

lab8：文件系统

南开大学 计算机学院和密码与网络空间安全学院

年级：2023级

专业：信息安全

姓名及学号：于昊汐（2312642）、陈安琪（2312481）、孙丽菲（2312724）

一、实验目的

1. 了解文件系统抽象层-VFS的设计与实现
2. 了解基于索引节点组织方式的Simple FS文件系统与操作的设计与实现
3. 了解“一切皆为文件”思想的设备文件设计
4. 了解简单系统终端的实现

二、实验内容

本次实验我们将完成练习、challenge，整体概述如下

练习0：填写已有实验

将实验2~7的代码整合到本实验并确保编译通过。

练习1：完成读文件操作的实现（需要编码）

在 `sfs_io_nolock()` 中实现文件的读取操作。

练习2：完成基于文件系统的执行程序机制的实现（需要编码）

改写 `proc.c` 中的 `load_icode` 等函数，实现基于文件系统的用户程序执行机制。

扩展练习 Challenge1：完成基于“UNIX的PIPE机制”的设计方案

设计并说明管道机制的数据结构和接口，处理同步互斥问题。

扩展练习 Challenge2：完成基于“UNIX的软连接和硬连接机制”的设计方案

设计并说明软连接和硬连接机制的数据结构和接口，处理同步互斥问题。

三、实验过程

练习0：填写已有实验

本实验依赖实验2/3/4/5/6/7。请把你做的实验2/3/4/5/6/7的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”/“LAB6”/“LAB7”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab8的测试应用程序，可能需对已完成的实验2/3/4/5/6/7的代码进行进一步改进。

ANS：正确将之前实验代码补充到对应位置，编译成功通过。

练习1：完成读文件操作的实现（需要编码）

首先了解打开文件的处理流程，然后参考本实验后续的文件读写操作的过程分析，填写在 `kern/fs/sfs/sfs_inode.c` 中的 `sfs_io_nolock()` 函数，实现读文件中数据的代码。

ANS：

`sfs_io_nolock()` 是 SFS 中最底层的文件数据读写函数，负责在 **不加锁的情况下**，将文件中从 `offset` 开始、长度为 `*alenp` 的数据，在 **磁盘块与内存缓冲区之间进行读写传输**。

练习1的核心任务，是补全 `sfs_io_nolock()` 中“真正执行文件数据读写”的核心逻辑，使内核能够 **正确读取磁盘中文件的数据**。具体而言，需要处理文件读写过程中 **偏移与磁盘块边界不对齐的问题**，将文件访问拆分为三种典型情况：起始非对齐块、中间完整对齐块、结束非对齐块，并分别调用 `sfs_rbuf` / `sfs_wbuf` 和 `sfs_rblock` / `sfs_wblock` 完成读写操作。

下面我们结合已有代码和Lab练习中完成的代码，分析 `sfs_io_nolock()` 的执行逻辑。

1. 参数合法性检查与读写边界处理

```
static int
sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, void *buf,
              off_t offset, size_t *alenp, bool write) {
    struct sfs_disk_inode *din = sin->din;
    assert(din->type != SFS_TYPE_DIR);
    off_t endpos = offset + *alenp, blkoff;
    *alenp = 0;
    // calculate the Rd/Wr end position
    if (offset < 0 || offset >= SFS_MAX_FILE_SIZE || offset > endpos) {
        return -E_INVALID;
    }
    if (offset == endpos) {
        return 0;
    }
    if (endpos > SFS_MAX_FILE_SIZE) {
        endpos = SFS_MAX_FILE_SIZE;
    }
    if (!write) {
        if (offset >= din->size) {
            return 0;
        }
        if (endpos > din->size) {
            endpos = din->size;
        }
    }
}
```

合法性检查：确保传入的文件偏移 `offset` 在合法范围 `[0, SFS_MAX_FILE_SIZE)` 内，并且没有溢出。

空操作判断：如果读写长度为 0，则直接返回成功。

边界裁剪：保证读写操作不超过文件最大长度；对于读操作，还需限制到实际文件大小 `din->size`。

通过 `*alenp = 0` 初始化已处理字节数，为后续累加做好准备。

2. 读写模式相关操作函数的选择

```

int (*sfs_buf_op)(struct sfs_fs *sfs, void *buf, size_t len,
                  uint32_t blkno, off_t offset);
int (*sfs_block_op)(struct sfs_fs *sfs, void *buf, uint32_t blkno,
                    uint32_t nblks);

if (write) {
    sfs_buf_op = sfs_wbuf, sfs_block_op = sfs_wblock;
}
else {
    sfs_buf_op = sfs_rbuf, sfs_block_op = sfs_rblock;
}

```

根据 `write` 标志选择操作函数：

- `sfs_buf_op`：用于处理块内读写（可能涉及偏移）。
 - `sfs_block_op`：用于处理完整块读写（块对齐，速度更快）。
- 这样可以统一循环中的逻辑，而不必在循环里频繁判断读写模式。

3. 初始化读写状态

```

int ret = 0;
size_t size, alen = 0;
uint32_t ino;
uint32_t blkno = offset / SFS_BLKSIZE; // The NO. of Rd/wr begin block
uint32_t nblks = endpos / SFS_BLKSIZE - blkno; // The size of Rd/wr blocks
blkoff = offset % SFS_BLKSIZE; // 块内偏移
size_t bytes;
uint32_t cur_blk = blkno;
off_t cur_off = offset;

```

初始化局部变量：

- `ret`：返回值标志。
- `alen`：累计已处理字节数。
- `blkno`：起始块号。
- `nblks`：涉及的完整块数。
- `blkoff`：起始偏移在块内的位置。
- `cur_blk` / `cur_off`：循环中当前处理的块号和文件偏移。

这些变量为块级循环读写提供基础。

4. 基于块的文件读写过程实现

```
while (cur_off < endpos)
```

循环条件保证当前处理的文件偏移 `cur_off` 小于预期的结束位置 `endpos`，即只处理指定长度的数据。

循环中会根据不同情况处理块内偏移、整块或尾块，直到完成整个读写操作。

情况一：起始偏移不对齐（部分块）

```

if (blkoff != 0) {
    ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);
    if (ret != 0) goto out;

    bytes = SFS_BLKSIZE - blkoff;
    if (cur_off + bytes > endpos) bytes = endpos - cur_off;

    ret = sfs_buf_op(sfs, (char *)buf + alen, bytes, ino, blkoff);
}

```

```

        if (ret != 0) goto out;

        alen += bytes;
        cur_off += bytes;
        cur_blk++;
        continue;
    }

```

分析:

(1) `if (blkoff != 0)`

判断当前起始偏移 `offset % SFS_BLKSIZE` 是否不在块边界上。

如果不对齐, 说明起始块只有一部分需要读写, 不能直接整块处理。

(2) `ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);`

获取逻辑块号 `cur_blk` 对应的磁盘块号 `ino`。

这里不涉及加锁, 适合单线程或已加锁的场景。

(3) `if (ret != 0) goto out;`

如果块映射失败, 返回错误。

(4) `bytes = SFS_BLKSIZE - blkoff;`

计算从当前偏移到块末尾的字节数, 即本块可以操作的最大长度。

(5) `if (cur_off + bytes > endpos) bytes = endpos - cur_off;`

如果本块剩余长度超过文件要求的读写长度, 则裁剪为剩余需要处理的字节数。

(6) `ret = sfs_buf_op(sfs, (char *)buf + alen, bytes, ino, blkoff);`

调用块内操作函数 (读或写), 从块内偏移 `blkoff` 开始, 处理 `bytes` 字节的数据。

数据从 `buf + alen` 开始写入或读取。

(7) `if (ret != 0) goto out;`

如果读写失败, 返回错误。

(8) `alen += bytes; cur_off += bytes; cur_blk++;`

累加已处理字节数。

更新当前文件偏移量。

移动到下一个块号。

(9) `continue;`

结束当前循环, 进入下一次循环处理下一个块。

情况二: 中间完整对齐块

```

if (endpos - cur_off >= SFS_BLKSIZE) {
    /*
     * (2) 处理整块对齐的完整块:
     * - 对每个完整块调用 sfs_bmap_load_nolock 获取磁盘块号 (ino)
     * - 使用 sfs_block_op (已指向 sfs_rblock 或 sfs_wblock) 传输整块数据
     * - 更新 alen、cur_off、cur_blk (可在循环中一次处理多块)
     */
    ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);
    if (ret != 0) goto out;

    ret = sfs_block_op(sfs, (char *)buf + alen, ino, 1);
    if (ret != 0) goto out;

    alen += SFS_BLKSIZE;
    cur_off += SFS_BLKSIZE;
    cur_blk++;
    continue;
}

```

分析

(1) `if (endpos - cur_off >= SFS_BLKSIZE)`

判断当前剩余字节是否至少为一个完整块大小。

如果是，则可以一次处理整块，效率更高。

(2) `ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);`

获取当前块的磁盘块号。

(3) `ret = sfs_block_op(sfs, (char *)buf + alen, ino, 1);`

调用整块读写函数（一次操作整块 `SFS_BLKSIZE` 大小）。

减少循环次数和偏移计算，提高效率。

(4) `alen += SFS_BLKSIZE; cur_off += SFS_BLKSIZE; cur_blk++;`

累加已处理字节数和文件偏移，进入下一个块。

(5) `continue;`

进入循环下一次迭代。

这样能高效处理中间对齐块，避免每次都处理块内偏移，优化读写性能。

情况三：末尾非对其块（尾块）

```
ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);
if (ret != 0) goto out;

bytes = endpos - cur_off;
ret = sfs_buf_op(sfs, (char *)buf + alen, bytes, ino, 0);
if (ret != 0) goto out;

alen += bytes;
cur_off += bytes;
break;
```

分析

(1) `ret = sfs_bmap_load_nolock(sfs, sin, cur_blk, &ino);`

获取最后一个块对应的磁盘块号。

(2) `bytes = endpos - cur_off;`

计算剩余需要读写的字节数（可能小于块大小）。

(3) `ret = sfs_buf_op(sfs, (char *)buf + alen, bytes, ino, 0);`

从块起始偏移 0 开始处理剩余字节。

使用块内操作函数保证不越界。

(4) `alen += bytes; cur_off += bytes;`

累加实际操作的字节数，更新文件偏移。

(5) `break;`

完成整个读写循环，退出 while。

5. 更新读写结果并维护 inode 元数据

```
*alenp = alen;
if (offset + alen > sin->din->size) {
    sin->din->size = offset + alen;
    sin->dirty = 1;
}
```

将实际读/写字节数返回给调用者（通过 `*alenp`）。

对写操作，如果文件大小发生变化，更新 `inode` 的 `size` 字段。

标记 `inode` 为脏（`dirty = 1`），在后续需要同步到磁盘时使用。

练习2: 完成基于文件系统的执行程序机制的实现（需要编码）

改写 `proc.c` 中的 `load_icode` 函数和其他相关函数，实现基于文件系统的执行程序机制。执行：`make qemu`。如果能看到 `sh` 用户程序的执行界面，则基本成功了。如果在 `sh` 用户界面上可以执行 `exit`, `hello`（更多用户程序放在 `user` 目录下）等其他放置在 `sfs` 文件系统中的其他执行程序，则可以认为本实验基本成功。

ANS:

首先实现 `load_icode` 函数：

`load_icode()` 的作用是：通过文件描述符 `fd`，从文件系统中读取一个 ELF 可执行文件，将其 TEXT / DATA / BSS 段正确加载到当前进程的用户虚拟地址空间中，建立用户栈，设置 `argc/argv`，并初始化 `trapframe`，使进程能够从用户态入口开始执行该程序。该函数的执行流程如下：

(1) 创建新的 mm 结构

```
struct mm_struct *mm = NULL;

if ((mm = mm_create()) == NULL) {return -E_NO_MEM;
}
```

为当前进程创建一个新的内存管理结构 `mm_struct`，用于记录进程的虚拟地址空间布局、页目录和各级信息。

如果系统内存不足或创建失败，则返回错误 `-E_NO_MEM`。

(2) 创建并初始化页目录（页表）

```
if (setup_pgdir(mm) != 0) {goto bad_pgdir_cleanup_mm;
}
```

为新进程分配并初始化页目录（页表），保证进程可以通过虚拟地址访问物理内存。

`mm->pgdir` 保存页目录的内核虚拟地址。

如果失败，需要清理之前分配的 `mm` 结构（跳转到 `bad_pgdir_cleanup_mm`）。

(3) 读取并解析 ELF 文件

(3.1) 读取 ELF 头

```
if ((ret = load_icode_read(fd, &elf, sizeof(elf), 0)) < 0)
```

从文件偏移 0 读取 ELF 文件头到内存。

ELF 文件头包含程序入口点、段表偏移、段数量等信息。

```
if (elf.e_magic != ELF_MAGIC) {
    ret = -E_INVALID ELF;
    goto bad_elf_cleanup_pgdir;
}
```

检查 ELF 魔数是否正确，确保文件是合法的 ELF 可执行文件。

(3.2) 遍历 Program Header（段表）

```

for (i = 0; i < elf.e_phnum; i++) {
    off_t phoff = elf.e_phoff + i * sizeof(struct proghdr);
    if ((ret = load_icode_read(fd, &ph, sizeof(ph), phoff)) < 0) {
        goto bad_cleanup_mmap;
    }

    if (ph.p_type != ELF_PT_LOAD)
        continue; // 非加载段跳过
}

```

遍历 ELF 文件中每个程序段头 (Program Header)。

通过 `load_icode_read()` 按需读取每个段头, 保证可以从磁盘文件逐段获取段信息, 而不一次性加载整个文件。

只处理可加载段 (`ELF_PT_LOAD`), 跳过其它类型 (如符号表、调试信息等)。

(3.3) 设置虚拟内存映射属性

```

uint32_t vm_flags = 0, perm = PTE_U | PTE_V;
if (ph.p_flags & ELF_PF_X) vm_flags |= VM_EXEC;
if (ph.p_flags & ELF_PF_W) vm_flags |= VM_WRITE;
if (ph.p_flags & ELF_PF_R) vm_flags |= VM_READ;
if (vm_flags & VM_READ) perm |= PTE_R;
if (vm_flags & VM_WRITE) perm |= (PTE_W | PTE_R);
if (vm_flags & VM_EXEC) perm |= PTE_X;

if ((ret = mm_map(mm, ph.p_va, ph.p_memsz, vm_flags, NULL)) != 0) {
    goto bad_cleanup_mmap;
}

```

根据段头中的权限标志 (可执行、可写、可读) 设置虚拟内存标记 `vm_flags` 和页表权限 `perm`。

调用 `mm_map()` 为段分配对应的虚拟地址空间 (VMA), 为后续页分配和内容加载做好准备。

(3.4) 分配页并加载 TEXT/DATA 段

```

uintptr_t start = ph.p_va;
uintptr_t file_end = ph.p_va + ph.p_filesz;
struct Page *page;
while (start < file_end) {
    uintptr_t la = ROUNDDOWN(start, PGSIZE);
    if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL)
        goto bad_cleanup_mmap;

    size_t off = start - la;
    size_t size = PGSIZE - off;
    if (file_end < la + PGSIZE)
        size = file_end - start;

    off_t file_off = ph.p_offset + (start - ph.p_va);
    if ((ret = load_icode_read(fd, page2kva(page) + off, size, file_off)) < 0)
        goto bad_cleanup_mmap;

    start += size;
}

```

按页 (Page) 分配内存，处理段的 TEXT/DATA 内容。

计算文件偏移与虚拟地址映射关系，确保从磁盘文件中按需加载正确的数据到新进程内存页。

处理跨页边界的情况，保证每页只填充有效数据。

(3.5) BSS 段初始化

```
start = ph.p_va + ph.p_filesz;
uintptr_t end = ph.p_va + ph.p_memsz;
while (start < end) {
    uintptr_t la = ROUNDDOWN(start, PGSIZE);
    if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL)
        goto bad_cleanup_mmap;

    size_t off = start - la;
    size_t size = PGSIZE - off;
    if (end < la + PGSIZE)
        size = end - start;

    memset(page2kva(page) + off, 0, size);
    start += size;
}
```

BSS 段不在文件中存储，需在内存中初始化为 0。

分页处理，确保跨页段也能正确零初始化。

(4) 建立用户栈

```
uint32_t stack_top = USTACKTOP;
vm_flags = VM_READ | VM_WRITE | VM_STACK;
if ((ret = mm_map(mm, stack_top - USTACKSIZE, USTACKSIZE, vm_flags, NULL)) != 0)
    goto bad_cleanup_mmap;

for (i = 0; i < 4; i++) // 分配4页用户栈
    assert(pgdir_alloc_page(mm->pgdir, stack_top - (i + 1) * PGSIZE, PTE_USER) !=
        NULL);
```

分配用户栈空间，建立用户栈虚拟内存区 (VMA)。

分页分配内存，保证进程有足够的栈空间运行。

(5) 切换当前进程地址空间

```
mm_count_inc(mm);
current->mm = mm;
current->pgdir = PADDR(mm->pgdir);
lsatp(PADDR(mm->pgdir)); // 刷新页表寄存器
```

将当前进程的内存管理结构切换到新创建的 mm。

更新页目录物理地址，并刷新页表寄存器 (SATP/CR3)，使 CPU 使用新的地址空间。

(6) 构造用户态 argc / argv

```
uintptr_t sp = stack_top;
uintptr_t uargv[EXEC_MAX_ARG_NUM + 1];
for (i = argc - 1; i >= 0; i--) {
    size_t len = strlen(kargv[i]) + 1;
    sp -= len;
}
```



```

    if (!copy_to_user(mm, (void *)sp, kargv[i], len)) {
        ret = -E_INVALID;
        goto bad_cleanup_mmap;
    }
    uargv[i] = sp;
    if (i == 0) break; /* avoid unsigned wrap of i */
}
/* align pointer */
sp = ROUNDDOWN(sp, sizeof(uintptr_t));
sp -= sizeof(uintptr_t) * (argc + 1);
/* copy argv pointers */
for (i = 0; i < argc; i++) {
    if (!copy_to_user(mm, (void *) (sp + i * sizeof(uintptr_t)), &uargv[i],
sizeof(uintptr_t))) {
        ret = -E_INVALID;
        goto bad_cleanup_mmap;
    }
}
/* argv[argc] = NULL */
uintptr_t zero = 0;
if (!copy_to_user(mm, (void *) (sp + argc * sizeof(uintptr_t)), &zero,
sizeof(uintptr_t))) {
    ret = -E_INVALID;
    goto bad_cleanup_mmap;
}

```

在用户栈中存放程序参数字符串和 argv 指针数组，支持带参数执行程序（类似 exec）。从栈顶向下分配空间，并按页对齐，保证用户程序访问正确。

argv[argc] 置 NULL，符合 C 标准约定。

(7) 初始化 trapframe (用户态入口)

```

struct trapframe *tf = current->tf;
uintptr_t sstatus = tf->sstatus;
memset(tf, 0, sizeof(struct trapframe));
tf->sstatus = (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE; // 设置用户态返回
tf->gpr.sp = sp; // 指向 argv 指针数组
tf->gpr.a0 = argc; // argc
tf->gpr.a1 = sp; // argv
tf->epc = elf.e_entry; // 程序入口

```

初始化当前进程的 trapframe，使进程从用户态入口执行。

sp 指向用户栈顶 (argv)，a0 / a1 保存 argc 和 argv。

设置 epc 为 ELF 程序入口地址。

修改 sstatus，清除 SPP（返回用户模式）并允许用户态中断（SPIE）。

(8) 错误处理与资源回收

```

bad_cleanup_mmap:
    exit_mmap(mm);
bad_elf_cleanup_pgdir:
    put_pgdir(mm);
bad_pgdir_cleanup_mm:
    mm_destroy(mm);
    goto out;

```

在加载过程中，如果任何步骤失败，需要回滚已分配的资源，避免内存泄漏。
包括释放页表、虚拟内存区和 mm 结构。

该函数与lab5的不同之处：

在 Lab5 中，`load_icode()` 的作用是 **将已经完整存在于内存中的 ELF 二进制程序加载为当前进程的用户程序**；而在 Lab8 中，`load_icode()` 的功能被进一步扩展为：**通过文件系统接口，从磁盘文件中按需读取 ELF 可执行文件内容，并完成用户进程的完整加载与初始化**。

具体变化有以下几点：

1、加载方式的根本变化（最核心区别）

Lab5 直接从内存拷贝 ELF 内容，不依赖文件系统，通过 `memcpy` 加载 TEXT/DATA 段；Lab8 则从文件系统读取，通过文件描述符 `fd` 调用 `load_icode_read()`，触发 `inode` 查找和磁盘块读入，实现程序从磁盘动态加载。

2、ELF 加载过程的差异

Lab5 可直接通过内存指针访问 ELF 头、段头和 TEXT/DATA 段；Lab8 则按需从磁盘读取 ELF 头和段头，通过计算文件偏移与虚拟地址映射，将每段数据逐页加载到进程内存，并处理跨页和跨块边界，实现真正的按需加载。

3、对文件系统的新增依赖

Lab5 完全不依赖文件系统；Lab8 则需要从 `fd` → `inode` → 磁盘块 读取数据，并调用 `sfs_io_nolock()`、`sfs_rblock/rbuf()`、`sfs_bmap_load_nolock()` 等接口，实现文件系统到进程执行的完整路径。

4、参数与栈初始化的增强

Lab5 仅分配用户栈并设置入口点；Lab8 在此基础上拷贝 `argc`、`argv` 和字符串到用户栈，并在 `trapframe` 中设置 `a0=argc`、`a1=argv`，支持用户程序访问命令行参数，实现完整 `exec` 功能。

`alloc_proc` 函数

```
// LAB8 add:
proc->filesp = NULL; // 初始化文件表指针为空
```

这里初始化进程的文件表指针为空，为后续创建文件表或继承父进程文件描述符做准备。

`dofork` 函数

```
//lab8 add
if (copy_files(clone_flags, proc) != 0)
{ // for LAB8
    goto bad_fork_cleanup_fs; // 回收文件结构相关资源
}
```

在创建子进程时复制父进程的文件表，如果复制失败则回收文件资源，保证子进程正确继承或独立文件描述符。

`put_kargv` 函数

```
static void
put_kargv(int argc, char **kargv) {
    if (kargv == NULL) return;

    for (int i = 0; i < argc; i++) {
        if (kargv[i] != NULL) {
            kfree(kargv[i]);
            kargv[i] = NULL; // 避免悬空指针
        }
    }
}
```

这里释放用户参数字符串后将指针置空，防止悬空指针和潜在的野指针访问。

最后，我们执行make和make qemu命令，编译代码并运行内核，结果如下图所示

OpenSBI v0.4 (Jul 2 2019 11:53:53)



The logo is an ASCII art representation of the text 'OpenSBI'. The 'O' is a large circle made of vertical and horizontal lines. The 'p' is a lowercase letter. The 'e' is a lowercase letter. The 'n' is a lowercase letter. The 'S' is a large letter. The 'B' is a large letter. The 'I' is a large letter. The logo is composed of various symbols like '|', '-', '/', and '\'. There is a small dot between the 'p' and 'e'.

```
Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base        : 0x80000000
Firmware Size        : 112 KB
Runtime SBI Version   : 0.1
```

```
PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffff (A,R,W,X)
(THU.CST) os is loading ...
```

Special kernel symbols:

```
entry 0xc020004a (virtual)
etext 0xc020b41c (virtual)
edata 0xc0291060 (virtual)
end    0xc0296910 (virtual)
```

Kernel executable memory footprint: 603KB

DTB Init

HartID: 0

DTB Address: 0x82200000

Physical Memory from DTB:

```
Base: 0x0000000080000000
Size: 0x0000000080000000 (128 MB)
End:  0x0000000087ffffff
```

DTB init completed

```
memory management: default_pmm_manager
physcial memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaoffset is 18446744070488326144
check_alloc_page() succeeded!
Page table directory switch succeeded!
Kernel stack guardians set succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
$ █
```

然后我们输入两个用户程序exit和hello进行验证，结果如下

```
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
I am the parent. Forking the child...
I am parent, fork a child pid 4
I am the parent, waiting now..
I am the child.
waitpid 4 ok.
exit pass.
Hello world!!.
I am process 5.
hello pass.
$ █
```

可以看到，这两个用户程序成功执行了，实验验证成功啦！

扩展练习 Challenge1：完成基于“UNIX的PIPE机制”的设计方案

如果要在ucore里加入UNIX的管道（Pipe）机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个（或多个）具体的C语言struct定义。在网络上查找相关的Linux资料和实现，请在实验报告中给出设计实现“UNIX的PIPE机制”的概要设方案，你的设计应当体现出对可能出现的同步互斥问题的处理。）

ANS:

一、设计目标

Pipe（管道）是一种进程间通信（IPC）机制，具有如下特征：

- 半双工通信：一个 pipe 有两个端点（读端和写端）
- 字节流传输：无消息边界，按字节顺序读写
- 内核缓冲区：数据仅存在于内核中

- 阻塞语义：空管道读阻塞，满管道写阻塞
- 引用计数：写端关闭→读返回 EOF；读端关闭→写产生错误

在 ucore 中，Pipe 应被设计为一种特殊文件对象，复用 VFS / file / fd 体系。

二、总体设计思路

用户态接口：pipe(), read(), write(), close()

内核接口：sys_pipe(), pipe_read(), pipe_write(), pipe_close()

核心原则：

- Pipe 不是磁盘文件
- 通过匿名 inode + file 表示
- 两个 fd 指向同一 pipe 内核对象
- 管道状态由引用计数 + 缓冲区状态决定
- 使用自旋锁与等待队列处理并发与阻塞

三、核心数据结构设计

Pipe 内核对象

```
#define PIPE_BUF_SIZE 4096

struct pipe {
    char buffer[PIPE_BUF_SIZE]; // 内核缓冲区
    size_t read_pos; // 读指针
    size_t write_pos; // 写指针
    size_t data_size; // 当前缓冲区数据量

    int readers; // 读端引用计数
    int writers; // 写端引用计数

    struct spinlock lock; // 保护 pipe 结构
    wait_queue_t read_wait; // 等待读数据的进程
    wait_queue_t write_wait; // 等待写空间的进程
};
```

Pipe inode (VFS 层)

```
struct pipe_inode {
    struct inode vfs_inode; // 嵌入通用 inode
    struct pipe *pipe; // 指向 pipe 对象
};
```

inode->i_op 指向 pipe 专用 file ops，不参与磁盘操作

file 结构 (复用 ucore)

```
struct file {
    ...
    off_t pos; // 对 pipe 无实际意义
    struct inode *node; // 指向 pipe_inode
    bool readable;
    bool writable;
};
```

pipe 的读写位置使用 pipe 内部的 `read_pos` / `write_pos`，而非 `file->pos`

四、关键接口设计

sys_pipe

```
int sys_pipe(int fd[2]);
```

创建 pipe 内核对象，分配两个 file

`fd[0]`：读端，`fd[1]`：写端

初始化 `readers=1`, `writers=1`

pipe_read

```
ssize_t pipe_read(struct file *file, void *buf, size_t len);
```

阻塞读取数据：空管道时，如果写端关闭返回 EOF，否则 sleep

拷贝 `min(len, data_size)` 字节

更新 `read_pos` / `data_size`

唤醒 `write_wait`

pipe_write

```
ssize_t pipe_write(struct file *file, const void *buf, size_t len);
```

阻塞写入数据：满管道时，如果读端关闭返回错误，否则 sleep

更新 `write_pos` / `data_size`

唤醒 `read_wait`

pipe_close

```
int pipe_close(struct file *file);
```

调整读写引用计数，读端关闭唤醒写端，写端关闭唤醒读端

`readers == 0 && writers == 0` 时释放 pipe

五、同步与互斥处理

- 并发读写：使用 spinlock + wait_queue，保证数据一致性与无死锁
- 阻塞与唤醒：空管道读阻塞，满管道写阻塞，读写完成唤醒对端
- EOF / SIGPIPE 语义：写端全部关闭 → 读返回 EOF；读端全部关闭 → 写返回错误

六、设计总结

本设计在 ucore 的 VFS 框架下，将 UNIX Pipe 机制实现为特殊匿名文件。通过共享 pipe 内核对象，实现多个 file 描述符之间的数据传递。使用自旋锁和等待队列保证并发访问安全和阻塞同步，并正确处理 EOF 与 SIGPIPE 等语义，实现行为与 UNIX/Linux Pipe 机制一致。

扩展练习 Challenge2：完成基于“UNIX的软连接和硬连接机制”的设计方案

如果要在 ucore 里加入 UNIX 的软连接和硬连接机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个（或多个）具体的 C 语言 struct 定义。在网络上查找相关的 Linux 资料和实现，请在实验报告中给出设计实现“UNIX 的软连接和硬连接机制”的概要设计方案，你的设计应当体现出对可能出现的同步互斥问题的处理。）

ANS:

一、背景知识

硬连接：硬连接是指多个目录项指向同一个 inode 节点。其特点为：

- 多个文件名指向同一个物理数据块
- 通过引用计数（`nlinks`）管理，只有当引用计数为 0 时才真正删除文件

- 不能跨文件系统，不能连接目录（防止循环）
- 删除其中一个连接不影响其他连接和数据本身

软连接：软连接是一个特殊的文件，其内容是另一个文件的路径名。其特点为：

- 其是一个独立的文件，有自己的 `inode`
- 文件内容存储目标文件的路径
- 可以跨文件系统，可以连接目录
- 目标文件删除后，软连接变为悬空连接

二、当前系统已实现部分

根据项目代码分析，当前 `ucore` 的 SFS 文件系统已有以下基础：

```
//磁盘inode结构 (sfs_disk_inode)
struct sfs_disk_inode {
    uint32_t size;           // 文件大小（字节）
    uint16_t type;           // 文件类型：SFS_TYPE_FILE, SFS_TYPE_DIR
    uint16_t nlinks;         // 硬链接计数（已存在但未使用）
    uint32_t blocks;         // 占用的块数
    uint32_t direct[SFS_NDIRECT]; // 直接块索引
    uint32_t indirect;       // 间接块索引
};
```

```
//文件类型定义
#define SFS_TYPE_INVALID    0    // 无效类型
#define SFS_TYPE_FILE      1    // 普通文件
#define SFS_TYPE_DIR       2    // 目录
#define SFS_TYPE_LINK      3    // 链接（已定义但未实现）
```

在磁盘 `inode` 结构（`sfs_disk_inode`）和文件类型定义中：

- `nlinks` 字段已经存在，为硬连接提供了基础
- `type` 字段定义了 `SFS_TYPE_LINK`（值为3），表明已预留软连接类型

所以，在现在的系统基础上只需要实现以下内容：

1. `nlinks` 字段未被正确维护和使用
2. `SFS_TYPE_LINK` 类型仅定义未实现
3. 缺少创建、删除连接的系统调用接口
4. 路径解析不支持软连接的递归跟踪
5. 缺少防止循环连接的机制

三、设计方案

3.1 数据结构

1. 软连接内容存储结构

软连接文件的数据块存储目标路径：

```
/* 软连接路径存储 */
#define SFS_MAX_SYMLINK_LEN    1024    // 软连接路径最大长度

struct sfs_symlink_data {
    char target_path[SFS_MAX_SYMLINK_LEN]; // 目标路径
};
```

2. 扩展 inode 元数据 (内存结构)

在 `sfs_inode` 中添加连接相关信息:

```
struct sfs_inode {
    struct sfs_disk_inode *din;      // 磁盘inode
    uint32_t ino;                    // inode号
    bool dirty;                       // 脏标记
    int reclaim_count;                // 回收计数
    semaphore_t sem;                  // inode信号量 (已存在)

    /* 新增字段用于连接管理 */
    bool is_symlink;                  // 是否为软连接
    char *symlink_target;              // 软连接目标路径 (缓存)

    list_entry_t inode_link;           // 链表节点
    list_entry_t hash_link;           // 哈希链表节点
};
```

3. 路径解析上下文结构

防止循环连接, 限制解析深度:

```
/* 路径解析上下文 */
#define MAX_SYMLINK_DEPTH    8      // 最大符号连接解析深度

struct path_resolve_context {
    int symlink_depth;               // 当前符号连接解析深度
    struct inode *visited_inodes[MAX_SYMLINK_DEPTH]; // 访问过的inode (检测循环)
};
```

3.2 接口设计

1. 硬连接接口

```
/**
 * sys_link - 创建硬连接
 * @old_path: 已存在文件的路径
 * @new_path: 新连接的路径
 *
 * 功能: 为old_path创建一个新的目录项new_path,
 *       两者指向同一个inode, nlinks加1
 *
 * 返回值: 成功返回0, 失败返回负错误码
 *
 * 错误处理:
 *   - ENOENT: 源文件不存在
 *   - EEXIST: 目标路径已存在
 *   - EISDIR: 不允许对目录创建硬连接
 *   - EXDEV: 跨文件系统连接
 */
int sys_link(const char *old_path, const char *new_path);

/**
 * sfs_link - SFS层硬连接实现
 * @sfs: sfs文件系统实例
 * @old_inode: 源文件inode
```



```

* @parent_dir: 目标目录inode
* @name: 新连接名称
*
* 实现步骤:
*   1. 检查old_inode类型（不允许目录）
*   2. 在parent_dir中创建新目录项，指向old_inode
*   3. 原子性增加old_inode->din->nlinks
*   4. 标记dirty并同步
*/
int sfs_link(struct sfs_fs *sfs, struct inode *old_inode,
             struct inode *parent_dir, const char *name);

/**
* sfs_unlink_update_nlinks - 删除连接时更新引用计数
* @sfs: sfs文件系统实例
* @inode: 要删除的inode
*
* 实现步骤:
*   1. 获取inode->sem信号量（保证原子性）
*   2. 减少nlinks计数
*   3. 如果nlinks == 0，释放数据块
*   4. 否则仅更新元数据
*   5. 释放信号量
*/
int sfs_unlink_update_nlinks(struct sfs_fs *sfs, struct inode *inode);

```

2. 软连接接口

```

/**
* sys_symlink - 创建软连接
* @target: 目标路径（可以不存在）
* @linkpath: 软连接路径
*
* 功能: 创建一个类型为SFS_TYPE_LINK的inode,
*       其数据块存储target路径字符串
*
* 返回值: 成功返回0, 失败返回负错误码
*/
int sys_symlink(const char *target, const char *linkpath);

/**
* sys_readlink - 读取软连接目标
* @path: 软连接路径
* @buf: 缓冲区
* @bufsize: 缓冲区大小
*
* 功能: 读取软连接指向的目标路径
* 返回值: 读取的字节数, 失败返回负错误码
*/
ssize_t sys_readlink(const char *path, char *buf, size_t bufsize);

/**
* sfs_create_symlink - SFS层软连接创建
* @sfs: sfs文件系统实例
* @parent_dir: 父目录inode
* @name: 连接名称
* @target: 目标路径

```

```

* @inode_store: 返回创建的inode
*
* 实现步骤:
*   1. 分配新inode, type设为SFS_TYPE_LINK
*   2. 分配数据块存储target路径
*   3. 设置size为路径长度
*   4. 在parent_dir中添加目录项
*/
int sfs_create_symlink(struct sfs_fs *sfs, struct inode *parent_dir, const char
*name, const char *target, struct inode **inode_store);

/**
* sfs_read_symlink - 读取软连接内容
* @inode: 软连接inode
* @buf: 输出缓冲区
* @bufsize: 缓冲区大小
*
* 返回值: 读取的字节数
*/
int sfs_read_symlink(struct inode *inode, char *buf, size_t bufsize);

```

3. 路径解析增强接口

```

/**
* vfs_lookup_follow - 支持软连接的路径解析
* @path: 要解析的路径
* @node_store: 返回最终inode
*
* 功能: 递归解析路径中的软连接, 自动跟踪
*
* 实现逻辑:
*   1. 初始化path_resolve_context
*   2. 逐级解析路径
*   3. 遇到软连接时:
*       a. 检查深度是否超过MAX_SYMLINK_DEPTH
*       b. 检查是否已访问过(防止循环)
*       c. 读取软连接目标路径
*       d. 递归解析目标路径
*   4. 返回最终inode
*
* 错误处理:
*   - ELOOP: 符号连接嵌套过深或循环
*   - ENOENT: 路径不存在(软连接悬空)
*/
int vfs_lookup_follow(const char *path, struct inode **node_store);

/**
* vfs_lookup_nofollow - 不跟踪软连接的路径解析
* @path: 要解析的路径
* @node_store: 返回inode(可能是软连接本身)
*
* 功能: 解析路径但不跟踪软, 用于删除、重命名软连接
*/
int vfs_lookup_nofollow(const char *path, struct inode **node_store);

/**
* is_symlink_loop - 检测符号连接循环

```

```

* @ctx: 路径解析上下文
* @inode: 当前inode
*
* 返回值: true表示存在循环
*/
bool is_symlink_loop(struct path_resolve_context *ctx, struct inode *inode);

```

3.3 系统调用层接口

需要在 `syscall.c` 和 `sysfile.c` 中添加:

```

/* 系统调用号定义 (kern/syscall/syscall.h) */
#define SYS_link      120    // 创建硬连接
#define SYS_unlink    121    // 删除连接 (已存在, 需增强)
#define SYS_symlink   122    // 创建软连接
#define SYS_readlink  123    // 读取软连接

/* 系统调用实现 (kern/fs/sysfile.c) */
int sysfile_link(const char *old_path, const char *new_path);
int sysfile_symlink(const char *target, const char *linkpath);
int sysfile_readlink(const char *path, char *buf, size_t bufsize);

```

四、同步互斥机制

1. 多级锁保护策略

```

/**
 * 锁的层次结构 (从高到低):
 * 1. fs_sem (sfs_fs->fs_sem)      - 文件系统级全局锁
 * 2. mutex_sem (sfs_fs->mutex_sem) - link/unlink操作互斥锁 (已存在)
 * 3. inode_sem (sfs_inode->sem)    - inode级锁
 */

/* 加锁顺序规则: 避免死锁 */
// 正确: 总是先锁文件系统, 再锁inode
void safe_link_operation() {
    lock_sfs_fs(sfs);           // 1. 获取文件系统锁
    down(&sfs->mutex_sem);       // 2. 获取link操作互斥锁
    down(&old_inode->sem);       // 3. 获取源inode锁
    down(&parent_dir->sem);      // 4. 获取目标目录锁

    // ... 执行连接操作 ...

    up(&parent_dir->sem);
    up(&old_inode->sem);
    up(&sfs->mutex_sem);
    unlock_sfs_fs(sfs);
}

```

2. 硬连接计数原子更新

```

/**
 * 原子性更新nlinks计数
 */
static inline void atomic_inc_nlinks(struct sfs_inode *sin) {
    down(&sin->sem);           // 获取锁

```

```

    sin->din->nlinks++;           // 增加计数
    sin->dirty = true;           // 标记脏
    up(&sin->sem);               // 释放锁
}

static inline int atomic_dec_nlinks(struct sfs_inode *sin) {
    down(&sin->sem);
    assert(sin->din->nlinks > 0); // 断言检查
    sin->din->nlinks--;
    int nlinks = sin->din->nlinks;
    sin->dirty = true;
    up(&sin->sem);
    return nlinks;
}

```

3. 软连接解析的并发控制

```

/**
 * 软连接解析中的竞争条件处理
 */
int vfs_lookup_follow_safe(const char *path, struct inode **node_store) {
    struct path_resolve_context ctx;
    ctx.symlink_depth = 0;

    struct inode *current_node = NULL;
    int ret;

    while (ctx.symlink_depth < MAX_SYMLINK_DEPTH) {
        // 解析当前路径
        ret = vfs_lookup_nofollow(path, &current_node);
        if (ret != 0) return ret;

        // 检查inode类型（需要加锁保护）
        uint32_t type;
        lock_inode(current_node); // 锁定inode
        ret = vop_gettype(current_node, &type);

        if (type != SFS_TYPE_LINK) {
            // 不是软连接，返回结果
            unlock_inode(current_node);
            *node_store = current_node;
            return 0;
        }

        // 是软连接，检查循环
        if (is_symlink_loop(&ctx, current_node)) {
            unlock_inode(current_node);
            return -E_LOOP;
        }

        // 读取软连接目标
        char target[SFS_MAX_SYMLINK_LEN];
        ret = sfs_read_symlink(current_node, target, sizeof(target));
        unlock_inode(current_node); // 读取后立即释放锁

        if (ret < 0) return ret;
    }
}

```

```
    // 记录访问过的inode
    ctx.visited_inodes[ctx.symlink_depth++] = current_node;

    // 继续解析目标路径
    path = target;
    vop_ref_dec(current_node);          // 释放引用
}

return -E_LOOP; // 深度超限
}
```

4. 关键临界区保护

操作	临界区	保护机制
创建硬连接	目录项添加 + nlinks 增加	mutex_sem + inode_sem
删除连接	目录项删除 + nlinks 减少 + 可能的块释放	mutex_sem + inode_sem
创建软连接	inode 分配 + 目录项添加	mutex_sem + fs_sem
读取软连接	读取 inode 数据块	inode_sem (读锁)
路径解析	多个 inode 访问	逐级加锁，立即释放

四、讨论

实验中重要的知识点及对应 OS 原理知识点

实验知识点	对应 OS 原理知识点	含义 / 关系 / 差异
VFS 抽象层设计与实现	文件系统抽象、系统调用接口	含义： VFS (Virtual File System) 是操作系统提供的抽象层，用于统一不同文件系统接口，使内核可以通过统一接口访问不同类型的文件系统。 关系： 实验中实现了 VFS 接口，统一管理 inode、file、fd，使文件操作与具体文件系统实现解耦，调用者不关心底层是 SFS 还是其他文件系统。 差异： 实验更注重接口和内核数据结构的具体实现，而 OS 原理中强调 VFS 的概念和作用，如多文件系统支持、缓存优化、通用系统调用封装。
Simple FS (SFS) 文件系统	索引节点 (inode) 文件组织	含义： SFS 是一种简化的文件系统，使用 inode 保存文件元信息（如权限、大小、块号映射）和数据块管理。 关系： 实验中 SFS 用 inode 管理文件，bmap 负责逻辑块到磁盘块映射，buf 管理缓存块，实现文件的读写和修改。 差异： 实验展示了 inode + bmap + buf 的实际工作机制，而 OS 原理中更多强调 inode 的概念、作用以及通用文件系统的设计思想，实验是具体实现例子。

实验知识点	对应 OS 原理知识点	含义 / 关系 / 差异
块级文件读写 (sfs_io_nolock)	文件读写原理、块设备 I/O	含义：文件 I/O 在 OS 中通过块或页为单位访问磁盘，通过缓冲区和缓存提高性能。 关系：实验实现了按块、偏移读写数据，并处理跨块情况，体现了文件读写逻辑。 差异：实验只处理直接读写和对齐问题，没有实现完整页缓存、写回策略 (write-back)、缓存一致性机制，这些是 OS 原理强调的性能优化点。
文件系统依赖的程序加载 (load_icode)	exec/程序加载机制、内存管理	含义：程序加载 (exec) 将磁盘上的可执行文件映射到用户虚拟地址空间，初始化 TEXT/DATA/BSS 段和用户栈。 关系：实验通过 load_icode 按页加载 ELF 文件，并分段映射到内存，实现程序执行准备。 差异：实验结合文件系统和内存管理，但未讲多级页表、段保护和完整的虚拟内存地址转换流程，这些是 OS 原理关注的底层机制。
用户栈和 argc/argv 初始化	用户态栈、进程参数传递	含义：用户栈在进程启动时需要分配空间，并放置程序参数、返回地址等数据，保证程序可以按调用约定运行。 关系：实验中将 argc、argv 拷贝到用户栈中，按页对齐，初始化用户栈指针，使程序能正确访问参数。 差异：实验展示了实际拷贝过程，而 OS 原理中讲的是栈初始化规范、调用约定和参数传递原理，更关注概念和一致性。
进程文件表初始化与复制 (alloc_proc / dofork)	进程描述符表、fork 文件描述符继承	含义：每个进程有文件表，记录打开的文件；fork 时子进程复制父进程文件表，保证资源共享或独立性。 关系：实验中创建进程时初始化文件表，fork 时复制父文件表，实现文件描述符继承。 差异：实验实现了具体操作和资源管理，而 OS 原理更强调 fork 的概念、引用计数和系统级一致性保证。
释放用户参数字符串 (put_kargv)	内存管理、资源回收	含义：程序参数字符串占用内存，使用完后必须释放，以防止内存泄漏或悬空指针。 关系：实验中通过 put_kargv 释放用户参数字符串，并置空指针，防止后续访问非法内存。 差异：实验是具体实现，OS 原理中强调内存安全、动态分配和防止野指针，是概念指导下的内存管理原则。
UNIX Pipe 机制设计 (Challenge1)	进程间通信 (IPC)	含义：Pipe 是进程间通信 (IPC) 机制，提供半双工的数据传输通道，内核维护缓冲区并处理同步。 关系：实验中 pipe 通过内核对象 + file 表实现，使用 spinlock 和 wait_queue 实现并发同步和阻塞/唤醒机制。 差异：实验展示了 pipe 的实现细节，而 OS 原理中 pipe 是 IPC 原语的一种，强调通用概念、原理和使用场景。
UNIX 软硬链接机制设计 (Challenge2)	文件系统目录与链接	含义：软链接和硬链接通过 inode 引用计数实现文件多路径访问，目录解析需要处理路径和同步。 关系：实验设计中考虑了 inode 引用计数、路径解析、锁机制，但未实现完整功能。 差异：实验体现设计思路，但缺少编码实现，OS 原理中更强调链接的概念、引用管理和目录树处理。

OS原理中重要但实验未覆盖的知识点

文件系统缓存与性能优化：实验直接读写磁盘块，未涉及页缓存、写回策略和缓存一致性。

进程间通信多样性：实验仅实现 pipe，未涉及信号量、消息队列、共享内存等 IPC 机制。

设备驱动与 I/O 原理：实验抽象设备文件，未涉及驱动模型、DMA 或中断驱动 I/O。

同步原语与并发控制：实验仅用 spinlock 处理 pipe 并发，未系统讲解死锁、互斥量和条件变量。

五、实验结论及心得体会

实验结论

1、成功实现基于文件系统的用户程序加载与执行

通过完成 `sfs_io_nolock()` 的文件读取逻辑，以及 `load_icode()` 函数的 ELF 加载、用户栈建立和 trapframe 配置，实现了从磁盘文件动态加载 ELF 可执行程序并切换到用户态执行。用户程序能够正确运行，并支持命令行参数访问，验证了 VFS、SFS 文件系统和内存管理模块的协作正确性。

2、完善了进程文件表与资源管理机制

在 `alloc_proc()` 初始化进程文件表，`dofork()` 复制父进程文件表，以及 `put_kargv()` 释放用户参数字符串后置空指针，确保了进程间文件描述符继承、独立性及资源安全回收，避免悬空指针和内存泄漏问题。

3、掌握了块级文件 I/O 与跨页/跨块处理逻辑

`sfs_io_nolock()` 能够正确处理起始偏移不对齐、整块处理和尾块不对齐三种情况，通过 `sfs_rbuf / sfs_wbuf` 和 `sfs_rblock / sfs_wblock` 实现高效的数据传输。实验加深了对块设备映射、磁盘块访问和文件系统底层读写逻辑的理解。

4、理解了进程间通信与文件系统扩展设计

在扩展练习中，设计了基于 UNIX 的管道机制（Pipe）和软/硬链接机制，明确了内核缓冲区、引用计数、阻塞/唤醒策略、同步互斥问题的处理方法。实验体现了 OS 中文件系统和进程间通信机制的设计思路，为后续扩展提供了实践基础。

5、验证了系统整体功能的正确性

通过 make 编译和 make qemu 测试，用户程序如 exit、hello 成功执行，证明实验功能完整、各模块协作正常，实验目标全部达成。

心得体会

1、加深了对操作系统分层设计的理解：用户程序通过文件系统接口访问资源，内核负责管理内存、磁盘和特权操作。实验强化了“一切皆为文件”思想及用户态/内核态隔离设计，使我们理解了 OS 如何在安全与灵活性之间取得平衡。

2、细节决定实现效果：文件读写跨块处理、ELF 段按页加载、用户栈及 argc/argv 初始化等操作都需精确处理，任何偏差都会导致程序加载失败或运行异常。这让我们深刻体会到 OS 底层实现的严谨性与“差之毫厘，谬以千里”的重要性。

3、培养了系统设计与优化意识：在文件读写、跨页加载和管道机制设计中，考虑了性能、内存和同步开销，使我们理解优秀 OS 设计不仅关注功能实现，还需兼顾资源利用率和系统效率。

4、团队协作提升了实验效率：实验涉及代码整合、功能实现和调试验证。通过合理分工，我们快速定位并解决文件读写异常、trapframe 配置错误和进程文件表继承问题，深刻体会到团队协作在复杂系统开发中的重要性。